GUÍA DE ESTUDIO DEL PROYECTO

Control Académico

Comprender el código, practicar y defender sus decisiones

TEMAS	PRÁCTICA	SOLUCIONARIO
16 temas esenciales	48 preguntas progresivas	48 respuestas al final

C++ · Algoritmos · Álgebra Lineal · Matemática Discreta

Preparada para el proyecto de Ingeniería en Sistemas UMG. Incluye la versión que limita los prerrequisitos a árboles binarios. Revisión del código: 24 de octubre de 2026.

Cómo utilizar esta guía

1. Lee los temas y localiza cada función en el proyecto.
2. Responde las preguntas sin consultar el solucionario.
3. Justifica tus respuestas con un ejemplo o un recorrido del código.
4. Compara al final y repite las preguntas que no puedas explicar.

Autoevaluación: 0 = no lo explico; 1 = lo reconozco; 2 = lo explico con ejemplo. Máximo orientativo: 96 puntos. La meta es comprender, no memorizar respuestas.

Ruta del documento: temas en páginas 3-6; preguntas en páginas 7-10; respuestas en páginas 11-16.

MAPA DEL PROYECTO

Qué estudiar y dónde

Los nombres de archivo son rutas relativas a la raíz del proyecto. La guía distingue el comportamiento actual de las mejoras propuestas.

Módulo	Responsabilidad principal
main.cpp	Inicio, carga de datos y menú principal.
config.h / modelos.h	Constantes, Curso y Estudiante.
estudiantes.cpp	Registro, búsqueda, consulta, modificación y eliminación.
cursos.cpp	Carga, consulta y selección del pensum; validación del bosque.
notas.cpp	Inscripción, componentes, estados, créditos y promedios.
archivos.cpp	Guardar y recuperar estudiantes y notas.
validaciones.cpp	Lectura segura, conversiones y separación de campos.
estadisticas.cpp	Resultados generales y por curso.
pantalla.cpp	Colores, encabezados, columnas, mensajes y pausas.
discreta.cpp	Submenú que conecta conjuntos, lógica y árboles.
conjuntos.cpp / logica.cpp	Operaciones de conjuntos y habilitación de cursos.
arboles.cpp	Izquierda/derecha, altura, descendientes y dibujo.
algebra.cpp	Matrices P/M, transpuesta, ponderación y puntos necesarios.
tests/arboles_binarios.cpp	Pruebas de estructura binaria y rechazo de datos inválidos.

Orden recomendado: fundamentos → datos y reglas → integración matemática → análisis crítico.

Preguntas: nivel 1, p. 7; nivel 2, p. 8; nivel 3, p. 9; nivel 4, p. 10. Respuestas: P01-P08, p. 11; P09-P16, p. 12; P17-P24, p. 13; P25-P32, p. 14; P33-P40, p. 15; P41-P48, p. 16.

TEMAS A ESTUDIAR / 1 DE 4

Fundamentos del programa

01. Flujo y menús

En el código: main.cpp, discreta.cpp, algebra.cpp y notas.cpp

Sigue el orden: preparar consola, cargar pensum, cargar estudiantes, pausar y entrar al menú. El switch elige una acción y el do-while repite mientras la opción sea distinta de cero.

Para dominarlo: Debes explicar qué función llama a cuál, cuándo se regresa de un submenú y por qué una variable global conserva su valor entre opciones.

02. Funciones y parámetros

En el código: estudiantes.h, cursos.h y sus implementaciones

Una función recibe información, realiza una tarea y puede devolver un resultado. int& permite modificar el contador original. const Estudiante& permite consultar al estudiante sin copiarlo y sin modificarlo mediante esa referencia.

Para dominarlo: Distingue declaración y definición, parámetros y argumentos, retorno y modificación por referencia. En estas firmas, los parámetros de arreglo no copian el arreglo completo.

03. Modelos y constantes

En el código: modelos.h y config.h

Curso agrupa código, nombre, créditos, ciclo y requisitos. Estudiante contiene datos personales, estados por curso y componentes de notas. Las constantes fijan 100 estudiantes, 50 cursos y cuatro componentes.

Para dominarlo: El proyecto usa funciones y struct; no es un diseño basado en clases con herencia. Los códigos se guardan como string para conservar ceros iniciales.

04. Arreglos e índices

En el código: modelos.h, estudiantes.cpp y notas.cpp

La posición c identifica el mismo curso en cursos[c], estado[c] y puntos[c][k]. El índice k identifica Parcial 1, Parcial 2, Zona o Final. Los índices empiezan en cero.

Para dominarlo: Separa capacidad máxima y cantidad ocupada. Al eliminar un estudiante se desplazan los posteriores y se reduce el contador; no se necesita borrar físicamente todo el arreglo.

TEMAS A ESTUDIAR / 2 DE 4

Datos, reglas y cálculos

05. Validación y lectura

En el código: validaciones.cpp

Las entradas se leen con getline y los números se interpretan con stringstream. Se comprueban formato, sobrantes y rango. Un nombre vacío o con el separador | se rechaza. El teclado exige un carné de la carrera 0900.

Para dominarlo: Prueba letras, números con texto añadido, límites, entradas vacías y fin de entrada. La validación del archivo y la del teclado todavía no aplican exactamente las mismas reglas.

06. Búsqueda e inicialización

En el código: estudiantes.cpp y cursos.cpp

Las búsquedas son secuenciales y devuelven la primera posición coincidente o -1. inicializarEstudiante deja todos los cursos en NO_CURSADO y todos sus componentes en SIN_CALIFICAR.

Para dominarlo: El -1 de una búsqueda significa no encontrado; el -1 de una nota significa sin calificar. Son convenciones diferentes. Nunca uses una posición -1 para indexar un arreglo.

07. Estados y notas

En el código: notas.cpp y config.h

Los máximos de los componentes son 10, 20, 35 y 35: se suman puntos netos, no se vuelve a aplicar un porcentaje. actualizarEstado deja EN_CURSO si falta un componente; con todos completos, aprueba desde 61.

Para dominarlo: Un cero es una nota real. Un aprobado histórico puede tener todos los componentes en -1 y sí aportar créditos. No debes recalcular ese caso como si fuera una inscripción incompleta.

08. Promedios y estadísticas

En el código: notas.cpp y estadisticas.cpp

El promedio simple del estudiante incluye únicamente cursos con cuatro componentes completos. No incluye notas parciales ni aprobaciones sin nota. Las estadísticas generales promedian los promedios de los estudiantes que tienen alguno.

Para dominarlo: Distingue nota acumulada, promedio simple, promedio ponderado y promedio de promedios. La clasificación global por promedio no implica haber aprobado todos los cursos.

TEMAS A ESTUDIAR / 3 DE 4

Persistencia y Discreta

09. Archivos y persistencia

En el código: archivos.cpp y datos/pensum.txt

El pensum tiene seis campos separados por |. El archivo de estudiantes contiene líneas E con datos personales y líneas C con curso, estado y cuatro notas. El guardado actual reemplaza el archivo completo.

Para dominarlo: Separar campos no basta: hay que validar rangos, relaciones entre datos y errores de escritura. Revisa cómo un dato guardado vuelve a convertirse en estructura al cargar.

10. Compilación y organización

En el código: compilar.ps1, archivos .h y .cpp

Los .h declaran estructuras y funciones compartidas; los .cpp contienen sus definiciones. #pragma once evita incluir varias veces la misma cabecera en una unidad de compilación. El enlazador reúne las definiciones compiladas.

Para dominarlo: Compila todos los .cpp de src. Ejecuta desde la raíz porque las rutas datos/pensum.txt y datos/estudiantes.txt son relativas al directorio de trabajo.

11. Conjuntos y proposiciones

En el código: conjuntos.cpp y logica.cpp

Cada conjunto es un arreglo bool: una posición indica pertenencia. Unión usa ||, intersección usa && y diferencia usa A[c] && !B[c]. La habilitación exige cuatro condiciones simultáneas.

Para dominarlo: Las condiciones son: no aprobado, no en curso, prerrequisito aprobado o inexistente, y créditos suficientes. Un curso reprobado puede volver a habilitarse si cumple los requisitos.

12. Árboles binarios y bosque

En el código: arboles.h, arboles.cpp y cursos.cpp

Cada curso tiene hasta un padre y dos hijos. El primer dependiente en el archivo es izquierdo; el segundo, derecho. HijosBinarios guarda ambos índices y usa -1 para una rama ausente. Las raíces forman un bosque.

Para dominarlo: No es un árbol binario de búsqueda ni se garantiza balance. La carga rechaza terceros hijos, ciclos, códigos duplicados y padres inexistentes; no descarta un tercer hijo para disimularlo.

TEMAS A ESTUDIAR / 4 DE 4

Recursión, Álgebra y pruebas

13. Recursión y recorridos

En el código: arboles.cpp

Una hoja tiene altura 1 y cero descendientes. La altura de un nodo es 1 más el máximo de las alturas de sus ramas; una rama vacía aporta 0. Los descendientes suman los de ambas ramas y los hijos presentes.

Para dominarlo: El dibujo visita nodo, izquierda y derecha: preorden. La recursión termina al no encontrar hijos. La altura cuenta cursos o niveles; solo aproxima semestres bajo las condiciones del modelo.

14. Matrices y álgebra

En el código: algebra.h y algebra.cpp

P contiene notas completas y M indica cuáles existen. Filas: estudiantes con al menos una nota completa del ciclo; columnas: cursos del ciclo. Una nota real de cero tiene M=1, mientras que una nota inexistente tiene M=0.

Para dominarlo: Estudia transpuesta, producto matriz-vector, promedio por curso con P^T y M^T, y promedio ponderado (P*c)/(M*c). El análisis admite hasta diez columnas por ciclo.

15. Ecuaciones y condiciones límite

En el código: algebra.cpp y notas.cpp

Para estimar lo necesario para aprobar: x = NOTA_MINIMA - acumulado. Compara x con cero y con los puntos máximos pendientes. Si x excede lo pendiente, ya no alcanza; si x no es positivo, basta con obtener cero.

Para dominarlo: Alcanzar 61 acumulados con componentes pendientes no cambia automáticamente el estado a APROBADO. La estimación numérica y el cierre de la evaluación cumplen funciones distintas.

16. Pruebas y consistencia

En el código: tests/arboles_binarios.cpp y módulos revisados

Prueba casos normales, fronteras y datos contradictorios. La versión actual ya restringe los árboles a dos hijos. Persisten puntos de estudio en edición de notas, coherencia de archivos y precisión del guardado.

Para dominarlo: Distingue comportamiento actual de mejora propuesta. Las preguntas avanzadas piden detectar problemas existentes y diseñar soluciones; no afirman que todas esas soluciones ya estén implementadas.

PREGUNTAS / DIFICULTAD PROGRESIVA

Nivel 1 · Reconocer

P01-P12 / Identifica conceptos, archivos y datos básicos.

P01. ¿Para qué sirve Control Académico y qué información principal administra?

Mi valoración: 0 / 1 / 2

P02. ¿Cuál es la función de entrada y qué debe cargar antes de mostrar el menú principal?

Mi valoración: 0 / 1 / 2

P03. ¿Qué diferencia hay entre un archivo .h y un archivo .cpp?

Mi valoración: 0 / 1 / 2

P04. ¿Qué es un struct y qué representan Curso y Estudiante?

Mi valoración: 0 / 1 / 2

P05. ¿Por qué conviene guardar "090008" como string y no como int?

Mi valoración: 0 / 1 / 2

P06. ¿Dónde se configuran las capacidades y la nota mínima? ¿Cuáles son sus valores actuales?

Mi valoración: 0 / 1 / 2

P07. Nombra los cuatro estados de un curso y explica qué significa cada uno.

Mi valoración: 0 / 1 / 2

P08. ¿Cuáles son los componentes de evaluación, sus máximos y el total posible?

Mi valoración: 0 / 1 / 2

P09. ¿Por qué se usa SIN_CALIFICAR = -1 en lugar de cero?

Mi valoración: 0 / 1 / 2

P10. ¿El promedio se guarda como campo de Estudiante? ¿Por qué?

Mi valoración: 0 / 1 / 2

P11. Si cursos[7] es Algoritmos, ¿qué representan e.estado[7] y e.puntos[7][FINAL]?

Mi valoración: 0 / 1 / 2

P12. ¿Por qué se utiliza do-while en el menú principal?

Mi valoración: 0 / 1 / 2

PREGUNTAS / DIFICULTAD PROGRESIVA

Nivel 2 · Explicar

P13-P24 / Sigue funciones, parámetros y cambios de información.

P13. ¿Por qué registrarEstudiante recibe int& numEstudiantes? ¿Qué pasaría sin &?

Mi valoración: 0 / 1 / 2

P14. Explica const Estudiante& e. ¿Qué permite y qué impide dentro de la función?

Mi valoración: 0 / 1 / 2

P15. ¿Cómo busca un estudiante buscarEstudiante() y qué significa devolver -1?

Mi valoración: 0 / 1 / 2

P16. ¿Por qué se llama a inicializarEstudiante() antes de usar un estudiante local nuevo?

Mi valoración: 0 / 1 / 2

P17. ¿Por qué "12abc" debe rechazarse al leer un entero, aunque empiece con dígitos?

Mi valoración: 0 / 1 / 2

P18. Si el arreglo admite 100 estudiantes pero hay tres registrados, ¿qué posiciones se recorren y cuál recibe el siguiente?

Mi valoración: 0 / 1 / 2

P19. En [Ana, Luis, Eva, Juan], ¿cómo queda el arreglo lógico al eliminar a Luis?

Mi valoración: 0 / 1 / 2

P20. ¿Qué guardan las líneas E y C del archivo de estudiantes y por qué importa su orden al cargar?

Mi valoración: 0 / 1 / 2

P21. ¿Por qué ejecutar desde src puede impedir encontrar el pensum aunque el ejecutable exista?

Mi valoración: 0 / 1 / 2

P22. ¿Qué provoca undefined reference to pausar() al compilar solo main.cpp y cómo se corrige?

Mi valoración: 0 / 1 / 2

P23. ¿Qué cuatro condiciones combina evaluarHabilitacion() y cómo influye una sola condición falsa?

Mi valoración: 0 / 1 / 2

P24. Un curso tiene componentes [10, 20, 35, -1]. ¿Qué total muestra notaCurso() y qué estado deja actualizarEstado()?

Mi valoración: 0 / 1 / 2

PREGUNTAS / DIFICULTAD PROGRESIVA

Nivel 3 · Resolver

P25-P36 / Calcula resultados y aplica los modelos matemáticos.

P25. Hay cursos con notas completas de 85 y 37, uno incompleto con 8 puntos y uno aprobado sin notas. ¿Cuál es el promedio y cuántos cursos incluye?

Mi valoración: 0 / 1 / 2

P26. Un curso de cuatro créditos tiene nota 80 y otro de seis créditos tiene 60. Calcula el promedio simple y el ponderado.

Mi valoración: 0 / 1 / 2

P27. Un estudiante lleva 8, 15 y 20 puntos; solo falta el final de 35. ¿Cuánto necesita para aprobar con 61 y qué porcentaje del final representa?

Mi valoración: 0 / 1 / 2

P28. Si acumula 20 puntos y solo falta el final de 35, ¿todavía puede llegar a 61? Justifica.

Mi valoración: 0 / 1 / 2

P29. Para P=[[80,0],[0,60]] y M=[[1,1],[0,1]], calcula el promedio de cada columna y explica los dos ceros de P.

Mi valoración: 0 / 1 / 2

P30. Obtén la transpuesta de [[1,2,3],[4,5,6]]. ¿Cómo cambian sus dimensiones?

Mi valoración: 0 / 1 / 2

P31. A1={Algoritmos, Álgebra} y A2={Álgebra, Discreta}. Obtén unión, intersección y A1-A2.

Mi valoración: 0 / 1 / 2

P32. Un curso está REPROBADO, tiene el prerrequisito aprobado y exige 70 créditos. El estudiante tiene 75. ¿Puede inscribirlo? ¿Y con 65 créditos?

Mi valoración: 0 / 1 / 2

P33. ¿Por qué el árbol actual es binario pero no es un árbol binario de búsqueda? ¿Cómo se asignan izquierda y derecha?

Mi valoración: 0 / 1 / 2

P34. A tiene hijos B y C; B tiene hijo D. Calcula altura de A, descendientes de A, hojas y recorrido preorden.

Mi valoración: 0 / 1 / 2

P35. ¿Por qué se habla de bosque y no de un único árbol? ¿Los mínimos de créditos se dibujan como enlaces padre-hijo?

Mi valoración: 0 / 1 / 2

P36. ¿Qué debe pasar si un pensum contiene un tercer hijo, un ciclo, un código duplicado o un padre inexistente? ¿Qué retorna cargarPensum()?

Mi valoración: 0 / 1 / 2

PREGUNTAS / DIFICULTAD PROGRESIVA

Nivel 4 · Analizar y defender

P37-P48 / Detecta límites, diseña pruebas y argumenta mejoras.

P37. Explica el recorrido desde registrar un estudiante hasta guardar y recuperar las notas de uno de sus cursos.

Mi valoración: 0 / 1 / 2

P38. Algoritmos está aprobado y Programación I inscrito. Se corrige Algoritmos y queda reprobado. ¿Qué inconsistencia permite hoy la edición y qué propondrías?

Mi valoración: 0 / 1 / 2

P39. Un archivo declara APROBADO y componentes [0,0,0,0]. ¿Por qué validar cada campo no basta? ¿Cómo respetarías aprobaciones históricas?

Mi valoración: 0 / 1 / 2

P40. ¿Puede un guardado cambiar una decisión alrededor de 61? Explica el riesgo y dos alternativas para controlarlo.

Mi valoración: 0 / 1 / 2

P41. ¿Por qué abrir el archivo correctamente no garantiza un guardado exitoso? Diseña un proceso que proteja la versión anterior.

Mi valoración: 0 / 1 / 2

P42. Ana tiene un curso con 100. Luis tiene nueve cursos con 50. Calcula el promedio de estudiantes y el de las diez notas. ¿Cuál usa el reporte general?

Mi valoración: 0 / 1 / 2

P43. ¿Contar descendientes es automáticamente O(n) por ser recursivo? Considera que hijosBinariosDe() busca hijos recorriendo el arreglo de cursos.

Mi valoración: 0 / 1 / 2

P44. Diseña pruebas para verificar que el bosque binario mantiene sus cursos y no entra en recursión infinita.

Mi valoración: 0 / 1 / 2

P45. ¿Qué casos probarías para aprobar desde 61 y distinguir notas pendientes de ceros reales?

Mi valoración: 0 / 1 / 2

P46. ¿Qué ventajas y limitaciones tienen los arreglos fijos? ¿Qué ocurre si un ciclo tiene once cursos en el análisis matricial actual?

Mi valoración: 0 / 1 / 2

P47. Propón tres simplificaciones que reduzcan repetición sin quitar contenidos de Algoritmos, Álgebra y Discreta.

Mi valoración: 0 / 1 / 2

P48. En una exposición, explica cómo se integran las tres materias y menciona dos límites reales de la versión actual.

Mi valoración: 0 / 1 / 2

SOLUCIONARIO / CONSULTAR DESPUÉS DE PRACTICAR

Respuestas P01-P08

Se aceptan otras explicaciones correctas si respetan el comportamiento y las reglas del proyecto.

P01. Objetivo del programa

Sirve para gestionar estudiantes y su avance en el pensum de Ingeniería en Sistemas. Administra datos personales, cursos, inscripciones, estados y componentes de notas. También ofrece reportes y aplicaciones de conjuntos, lógica, árboles binarios y matrices.

P02. Inicio de ejecución

La entrada es main(). Primero prepara la consola y llama a cargarDatos(), que carga el pensum y luego los estudiantes guardados. Si no puede continuar con el pensum, termina con error. Después muestra el menú dentro de un ciclo.

P03. Cabecera e implementación

La cabecera .h expone las declaraciones de funciones, estructuras o constantes compartidas. El .cpp suele contener las definiciones de las funciones. Incluir una cabecera permite conocer una firma, pero no reemplaza compilar y enlazar su implementación.

P04. Estructuras de datos

Un struct agrupa campos relacionados bajo un tipo. Curso describe un elemento del pensum. Estudiante contiene datos personales y arreglos con el estado y los componentes de cada curso. Se accede a sus campos con el operador punto.

P05. Códigos como texto

Porque el código es un identificador, no una cantidad para operar. Como texto conserva el cero inicial y puede compararse exactamente con el archivo del pensum. Al convertirlo a un entero se perdería ese formato.

P06. Constantes del proyecto

En config.h. MAX_ESTUDIANTES vale 100, MAX_CURSOS vale 50, NUM_COMPONENTES vale 4 y NOTA_MINIMA vale 61. Centralizarlos permite consultar y cambiar esas reglas sin repetir números por todo el programa.

P07. Estados académicos

NO_CURSADO (0): no registrado como llevado. EN_CURSO (1): inscrito y todavía sin evaluación completa. APROBADO (2): aprobado por notas o registrado como aprobación histórica sin notas. REPROBADO (3): evaluación completa con nota inferior al mínimo.

P08. Componentes de nota

Parcial 1: 10 puntos; Parcial 2: 20; Zona: 35; Final: 35. El total es 100. El programa suma los puntos netos ingresados; no multiplica nuevamente esos valores por porcentajes.

SOLUCIONARIO / CONSULTAR DESPUÉS DE PRACTICAR

Respuestas P09-P16

Se aceptan otras explicaciones correctas si respetan el comportamiento y las reglas del proyecto.

P09. Ausencia frente a cero

Porque cero es un resultado válido de una evaluación. El -1 identifica que la evaluación todavía no tiene nota. Esa diferencia permite distinguir entre un componente pendiente y uno ya calificado con cero.

P10. Promedio calculado

No. Se calcula a partir de las notas cada vez que se necesita. Así se evita mantener un campo duplicado que podría quedar desactualizado después de corregir los componentes de un curso.

P11. Relación entre índices

e.estado[7] es la situación de ese estudiante en Algoritmos. e.puntos[7][FINAL] es su puntaje del examen final en ese mismo curso. El índice del curso debe conservar el mismo significado en todos los arreglos relacionados.

P12. Repetición del menú

Porque ejecuta el cuerpo al menos una vez y comprueba la condición después. El usuario necesita ver el menú y elegir antes de decidir si se repite. El ciclo continúa mientras la opción sea distinta de cero.

P13. Paso por referencia

Con & modifica el contador original del llamador. Sin & recibiría una copia: incrementarla dentro de la función no actualizaría el contador del menú. Eso podría hacer que el siguiente registro reutilizara una posición ya ocupada.

P14. Referencia constante

Recibe una referencia al estudiante original y evita copiar la estructura completa. const impide modificar sus campos mediante esa referencia. Es apropiado para consultas y cálculos que deben leer información sin cambiarla.

P15. Búsqueda secuencial

Compara el carné con los elementos entre cero y numEstudiantes-1. Devuelve la primera posición coincidente; -1 significa que no encontró ninguna. Antes de usar la posición como índice, el llamador debe comprobar ese resultado.

P16. Inicialización segura

Los arreglos numéricos de un objeto local sin inicialización pueden tener valores indeterminados. La función asigna NO_CURSADO a los estados y SIN_CALIFICAR a los componentes. No basta con inicializar los nombres, y llenar todo con cero daría notas falsas.

SOLUCIONARIO / CONSULTAR DESPUÉS DE PRACTICAR

Respuestas P17-P24

Se aceptan otras explicaciones correctas si respetan el comportamiento y las reglas del proyecto.

P17. Lectura estricta

La conversión puede leer el 12 y dejar abc pendiente. El código comprueba que no quede otro carácter distinto de espacios después del número. Por eso se valida la línea completa y no solo su prefijo numérico.

P18. Capacidad y ocupación

Se recorren las posiciones 0, 1 y 2. El siguiente se escribe en estudiantes[3] y el contador pasa de 3 a 4. La capacidad limita el almacenamiento; numEstudiantes indica cuántos elementos son válidos en ese momento.

P19. Eliminación sin huecos

Eva se copia sobre Luis y Juan sobre la posición anterior de Eva. El arreglo lógico queda [Ana, Eva, Juan] y el contador baja de 4 a 3. El contenido sobrante fuera del rango ocupado no debe recorrerse ni guardarse.

P20. Formato de persistencia

E guarda carné, nombre y carrera. C guarda carné, código de curso, estado y cuatro componentes. El cargador actual busca al estudiante al leer cada C, por lo que su línea E debe aparecer antes. El guardador ya produce ese orden.

P21. Directorio de trabajo

Porque datos/pensum.txt es una ruta relativa al directorio de trabajo, no a la ubicación de main.cpp. Desde src apuntaría a src/datos/pensum.txt. Debe ejecutarse desde la raíz; el script con -Ejecutar cambia a esa carpeta.

P22. Errores de enlace

main.cpp conoce la declaración mediante pantalla.h, pero el enlazador necesita la definición de pantalla.cpp y los demás módulos usados. Desde la raíz se compila todo con: g++ src/*.cpp -o control.exe. Después se ejecuta desde esa raíz.

P23. Conjunción de requisitos

Combina: el curso no está aprobado; no está en curso; su prerrequisito está aprobado o no existe; y hay créditos suficientes. Las une con &&. Una sola condición falsa hace falsa la habilitación y el curso no se ofrece para inscribir.

P24. Acumulado con pendientes

notaCurso() suma los componentes calificados y devuelve 65. actualizarEstado() deja EN_CURSO porque falta el final. Aunque el acumulado supera 61, el código exige los cuatro componentes completos para decidir aprobación o reprobación por notas.

SOLUCIONARIO / CONSULTAR DESPUÉS DE PRACTICAR

Respuestas P25-P32

Se aceptan otras explicaciones correctas si respetan el comportamiento y las reglas del proyecto.

P25. Promedio con exclusiones

El promedio es (85 + 37) / 2 = 61 e incluye dos cursos. El incompleto no entra, y tampoco el aprobado histórico sin componentes. Este último sí puede aportar créditos porque el conteo de créditos depende del estado APROBADO.

P26. Ponderación por créditos

Simple: (80 + 60) / 2 = 70. Ponderado: (80*4 + 60*6) / (4 + 6) = 680/10 = 68. El segundo curso influye más porque tiene más créditos. Ambos cálculos consideran notas completas.

P27. Puntos necesarios

Acumula 8 + 15 + 20 = 43. Necesita 61 - 43 = 18 puntos de los 35 disponibles. Eso equivale a 18/35*100, aproximadamente 51.43 % del final. El valor requerido está dentro del rango posible.

P28. Aprobación imposible

No. Necesitaría 61 - 20 = 41 puntos, pero solo puede obtener 35. Incluso con el máximo terminaría con 55. La comprobación correcta compara lo requerido con el máximo pendiente, además de calcular la diferencia.

P29. Matriz indicadora

Primera columna: suma 80 y cantidad 1, promedio 80. Segunda: suma 60 y cantidad 2, promedio 30. P[0][1]=0 es una nota real porque M=1. P[1][0]=0 indica ausencia de nota porque M=0; no entra al promedio.

P30. Transpuesta y dimensiones

La transpuesta es [[1,4],[2,5],[3,6]]. La matriz original es 2x3 y la transpuesta es 3x2. Cada elemento A[i][j] pasa a AT[j][i]. No se cambian los valores: se intercambian filas y columnas.

P31. Operaciones de conjuntos

Unión: {Algoritmos, Álgebra, Discreta}. Intersección: {Álgebra}. A1-A2: {Algoritmos}. La unión tiene 3 elementos y se verifica inclusión-exclusión: 2 + 2 - 1 = 3. La diferencia no es simétrica.

P32. Reinscripción

Con 75 puede: no está aprobado ni en curso y cumple prerrequisito y créditos. Con 65 no puede porque falla la condición de créditos. Si lo reinscribe, el comportamiento actual borra sus componentes anteriores y conserva solo el intento más reciente.

SOLUCIONARIO / CONSULTAR DESPUÉS DE PRACTICAR

Respuestas P33-P40

Se aceptan otras explicaciones correctas si respetan el comportamiento y las reglas del proyecto.

P33. Binario frente a búsqueda

Es binario porque cada nodo admite como máximo dos hijos. No usa comparaciones de claves para imponer menores a la izquierda y mayores a la derecha. El primer dependiente en el archivo queda a la izquierda y el segundo a la derecha; los enlaces significan prerrequisitos.

P34. Altura y preorden

La altura de A es 3: A-B-D. Tiene 3 descendientes: B, C y D. Las hojas son C y D. Si B es izquierdo y C derecho, el preorden es A, B, D, C. La altura de una hoja se cuenta como 1 en este proyecto.

P35. Bosque y reglas de créditos

Hay varios cursos sin prerrequisito, por lo que existen varias raíces y árboles independientes. El conjunto forma un bosque binario. Los mínimos de créditos son restricciones adicionales de habilitación; no se convierten en enlaces de parentesco en este modelo.

P36. Validación binaria

La validación rechaza el pensum y muestra el motivo. cargarPensum() devuelve -2 para una estructura inválida y el inicio impide continuar. No se ocultan hijos. El valor -1 se reserva para no poder abrir el archivo; cero cursos tiene otro tratamiento.

P37. Recorrido completo de datos

Se valida el carné, se inicializa el estudiante y se agrega al arreglo. Luego se inscribe un curso habilitado y se ingresan sus componentes; se actualiza el estado. Guardar genera E y C. Al cargar, se busca carné y código para reconstruir sus posiciones y datos.

P38. Dependencias tras corregir

La edición actual cambia el estado de Algoritmos sin revisar el curso dependiente, por lo que Programación I sigue inscrito sin cumplir el prerrequisito. Propongo validar el cambio antes de aplicarlo y rechazarlo o pedir resolver primero las dependencias. La validación binaria no corrige esta regla académica.

P39. Estado y nota importados

Los campos aislados son válidos, pero el conjunto es contradictorio. Se debe comprobar que las notas completas produzcan el estado esperado. Hay que distinguir el aprobado histórico con todos los componentes en -1; recalcular indiscriminadamente todos los estados destruiría ese caso permitido.

P40. Precisión al persistir

Sí. La precisión predeterminada puede redondear componentes y cambiar el total al recargar; el estado almacenado puede quedar en desacuerdo. Se puede definir una escala fija de centésimas y operar con enteros, o serializar double con suficiente precisión, además de validar coherencia.

SOLUCIONARIO / CONSULTAR DESPUÉS DE PRACTICAR

Respuestas P41-P48

Se aceptan otras explicaciones correctas si respetan el comportamiento y las reglas del proyecto.

P41. Guardado seguro

Pueden fallar la escritura o el cierre. El guardador actual devuelve true sin comprobarlos. Una mejora es escribir en un temporal, verificar escritura y cierre, y después reemplazar el archivo original mediante una operación segura. Si falla, debe conservarse el original e informarse el error.

P42. Promedio de promedios

El promedio de estudiantes es (100 + 50)/2 = 75. El de las diez notas es (100 + 9*50)/10 = 55. El reporte general actual usa 75 porque da el mismo peso a cada estudiante con promedio, aunque tengan distintas cantidades de cursos.

P43. Costo de la recursión

No. Cada nodo visitado puede exigir recorrer hasta n cursos para localizar sus hijos. Al visitar n nodos, el costo puede ser O(n^2). Con enlaces precalculados podría ser O(n), pero para 50 cursos la implementación actual es pequeña y fácil de seguir.

P44. Pruebas del árbol

Incluye hoja, cadena de un hijo, dos ramas y varias raíces. Comprueba altura, descendientes y que las raíces cubran todos los nodos. Rechaza tres hijos, autorreferencia, ciclo de varios nodos, padre inexistente y códigos repetidos. Usa archivos de prueba separados del pensum real.

P45. Fronteras de evaluación

Probaría totales completos justo inferiores a 61, exactamente 61 y superiores; cuatro ceros completos; un componente -1 aunque el acumulado supere 61; y aprobaciones históricas sin notas. También comprobaría que la interfaz rechace valores mayores al máximo y negativos distintos de -1.

P46. Límites de arreglos

Son sencillos y hacen visibles los límites, pero obligan a controlar capacidad e índices. El análisis matricial admite diez columnas y hoy omite las restantes sin avisar. Se puede detectar y rechazar ese caso o ampliar la representación, sin sustituir obligatoriamente los arreglos del proyecto.

P47. Simplificación razonada

Separar cálculos y cambios de estado de los menús; reutilizar la limpieza de componentes y la validación de cambios; y reducir comentarios que repiten literalmente las instrucciones. Conservaría matrices, conjuntos y árboles por su función académica, con nombres claros y comentarios sobre sus reglas.

P48. Defensa integral

Algoritmos aporta menús, búsqueda, validación y archivos; Discreta aporta conjuntos, proposiciones y bosque binario; Álgebra aporta matrices, ponderación y ecuaciones. Entre los límites están no conservar intentos anteriores y no revisar dependencias al corregir notas. Además, el bosque no garantiza balance ni organiza por claves.